

COMP3015 CW1 AI Usages

I wish to separate textures using the current TextureInfo structure, how can I update my bindings to pass the separate diffuse and normal texture for each

To support different textures (e.g., floor and wall diffuse/normal) without adding more bindings or uniforms, you should use the same TextureInfo struct and update the texture unit bindings and uniform values in your C++ code before each draw call. This is the standard and efficient way in OpenGL.

Key idea:

You only need two sampler uniforms (diffuse and normal). Before drawing each object, bind the correct textures to the texture units (e.g., 0 and 1), and set the uniforms to use those units. The shader code does not need to change.

How to do it

1. In your C++ render code

// For floor

```
glActiveTexture(GL_TEXTURE0);
```

```
glBindTexture(GL_TEXTURE_2D, floorDiffuseTexture);
```

```
glActiveTexture(GL_TEXTURE1);
```

```
glBindTexture(GL_TEXTURE_2D, floorNormalTexture);
```

```
prog.setUniform("Textures.diffuseTexture", 0); // Use texture unit 0
```

```
prog.setUniform("Textures.normalTexture", 1); // Use texture unit 1
```

// Draw floor

```
plane.render();
```

// For wall

```
glActiveTexture(GL_TEXTURE0);
```

```

glBindTexture(GL_TEXTURE_2D, wallDiffuseTexture);

glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, wallNormalTexture);

prog.setUniform("Textures.diffuseTexture", 0); // Use texture unit 0
prog.setUniform("Textures.normalTexture", 1); // Use texture unit 1

// Draw wall
wall.render();

```

Mixing Textures after normal mapping

Currently I apply normal mapping within my fragment shader through main, how can I apply normal mapping with my mix texture before mixing them?

To apply the normal map to your mixDiffuseTexture before mixing, you should also mix the normal maps in the same way as the diffuse textures. This allows the blended region to use the blended normal for lighting.

Steps:

1. Sample both normal maps (normalTexture and mixNormalTexture).
2. Convert both from [0,1] to [-1,1].
3. Mix them using the alpha of mixDiffuseTexture.
4. Use the mixed normal in your lighting calculation.

Here's how to update your shader:

```

vec3 blinnPhong(LightInfo light, vec3 position, vec3 normal, vec3 texColor) {

    vec3 ambient = light.La * texColor;

    vec3 s = normalize(light.Position.xyz - position);

    float sDotN = max(dot(s, normal), 0.0);

```

```

vec3 diffuse = light.Ld * texColor * sDotN;
vec3 specular = vec3(0);
if (sDotN > 0.0) {
    vec3 v = normalize(-position);
    vec3 h = normalize(s + v);
    specular = Material.Ks * pow(max(dot(h, normal), 0.0), Material.Shininess);
}
return ambient + (diffuse + specular) * light.L;
}

```

```

void main()
{
    vec3 color = vec3(0);

    // Sample textures
    vec4 baseTex = texture(Textures.diffuseTexture, TexCoord);
    vec4 mixTex = texture(Textures.mixDiffuseTexture, TexCoord);

    // Sample normal maps and convert to [-1,1]
    vec3 baseNorm = texture(Textures.normalTexture, TexCoord).rgb * 2.0 - 1.0;
    vec3 mixNorm = texture(Textures.mixNormalTexture, TexCoord).rgb * 2.0 - 1.0;

    // Mix diffuse and normal using mixTex alpha
    float blend = mixTex.a;
    vec3 diffuseTex = mix(baseTex.rgb, mixTex.rgb, blend);
    vec3 normalTex = normalize(mix(baseNorm, mixNorm, blend));

    // Lighting

```

```

for (int i = 0; i < NumLights; i++) {
    color += blinnPhong(Lights[i], Position, normalTex, diffuseTex);
}
FragColor = vec4(color, 1.0);
}

```

Mixing normals

What am I missing for my mixing textures, it runs but doesn't apply correctly

```

void main()
{
    vec3 color = vec3(0);

    // Normal mapping

    vec3 diffuseTex = texture(Textures.diffuseTexture, TexCoord).rgb;
    vec3 normalTex = texture(Textures.normalTexture, TexCoord).rgb;
    vec3 mixDiffuseTex = texture(Textures.mixDiffuseTexture, TexCoord).rgb;
    vec3 mixNormalTex = texture(Textures.mixNormalTexture, TexCoord).rgb;
    normalTex = 2.0 * normalTex - 1.0;
    mixNormalTex = 2.0 * mixNormalTex - 1.0;
    vec3 finalDiffuseTex = mix(diffuseTex, mixDiffuseTex.rgb, mixDiffuseTex.a);
    vec3 finalNormalTex = mix(normalTex, mixNormalTex, mixDiffuseTex.a);

    // Lighting

    for (int i = 0; i < NumLights; i++) {
        color += blinnPhong(Lights[i], Position, normalize(finalNormalTex), finalDiffuseTex);
    }

    FragColor = vec4(color, 1.0);
}

```

● The bug is here

```
vec3 mixDiffuseTex = texture(Textures.mixDiffuseTexture, TexCoord).rgb;
```

...

```
vec3 finalDiffuseTex = mix(diffuseTex, mixDiffuseTex.rgb, mixDiffuseTex.a);
```

🔗 **mixDiffuseTex is a vec3. It has no .a.**

So your blend factor is **undefined / garbage / optimized away**.

That's why the mix looks wrong or invisible even though the shader runs.

Can you update my new light uniforms for a orange fire colour inside the barrel object

```
prog.setUniform("Lights[2].L", vec3(0.6f)); // Light intensity
prog.setUniform("Lights[2].La", vec3(0.6f)); // Ambient light intensity
prog.setUniform("Lights[2].Ld", vec3(0.9f)); // Diffuse light intensity
```

Set fire light properties (initScene)

```
// Fire light (inside barrel)
prog.setUniform("Lights[2].La", vec3(0.2f, 0.05f, 0.0f)); // warm ambient
prog.setUniform("Lights[2].Ld", vec3(1.0f, 0.4f, 0.1f)); // orange fire
prog.setUniform("Lights[2].L", vec3(1.0f)); // base intensity
```

This already makes it look “fire-like”.

Move light position

```
vec4 firePos = vec4(
    0.0f, // same X as barrel
    height, // subtle vertical movement
    4.0f, // same Z as barrel
    1.0f
);
```

// set fire light

```
prog.setUniform("Lights[2].Position", firePos);
```

GLSL Input

How does the rendering pipeline work within GLSL for keyboard input controls, does it require GLSL to work?

1. The CPU handles input

The CPU is in charge of everything related to the system:

- Keyboard keys (W, A, S, D)
- Mouse movements
- Window events

In your C++ code, you would do something like:

```
if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) {  
    cameraPos += cameraSpeed * deltaTime;  
}  
  
if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS) {  
    cameraPos -= cameraSpeed * deltaTime;  
}
```

Here, cameraPos is a glm::vec3 representing the camera's location in the scene.

2. Pass data to GLSL using uniforms

GLSL **cannot directly read input**, but it **can read variables you pass from the CPU**. These are called **uniforms**.

Example:

// C++ side

```
prog.setUniform("viewMatrix", view);  
prog.setUniform("projectionMatrix", projection);  
prog.setUniform("cameraPos", cameraPos);
```

Then in your GLSL vertex or fragment shader:

// GLSL side

```
uniform mat4 viewMatrix;
```

```
uniform mat4 projectionMatrix;
```

```
uniform vec3 cameraPos;
```

Now your shader **knows the camera position** and can use it for:

- Lighting calculations
 - View-dependent effects
 - Reflections, fog, etc.
-

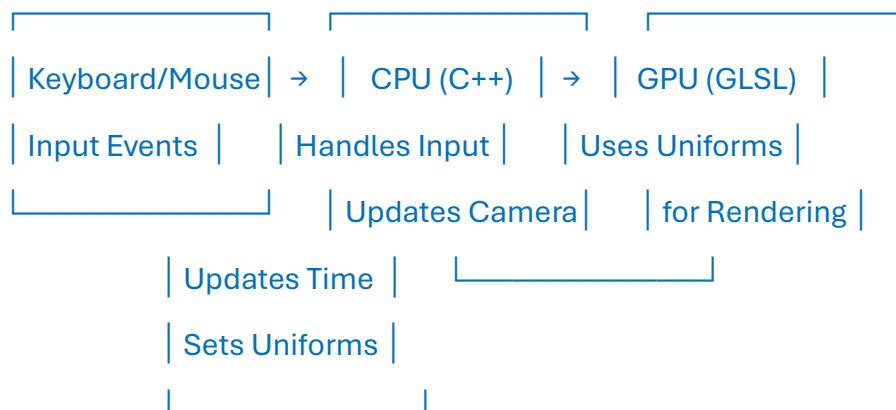
3. Rendering loop

Your CPU main loop would look like this:

```
while (!glfwWindowShouldClose(window)) {  
    float t = glfwGetTime();  
  
    // 1. Update input and camera  
    updateInput(window);    // modifies cameraPos  
    updateScene(t);        // updates animation variables  
  
    // 2. Send updated uniforms to GPU  
    prog.setUniform("viewMatrix", glm::lookAt(cameraPos, target, up));  
  
    // 3. Draw  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
    scene.render();  
  
    glfwSwapBuffers(window);  
    glfwPollEvents();  
}
```

Notice that **the shader never sees the keyboard directly**—it only sees the **updated uniforms from the CPU**.

Diagram of the flow



☑ Key takeaway:

- GLSL **cannot interact with the system.**
- All interactivity **must go through the CPU.**
- GLSL can only read what you **pass as uniforms or attributes.**

Getting GLFW window in scenebasic_uniform

I need GLFWwindow for userInput() in scenebasic_uniform, but I cannot get the window from scenerunner.h into scenebasic_uniform.cpp

Ah! I see exactly what's happening. The core issue is you want your SceneBasic_Uniform class to handle GLFW input, but Scene (the base class) doesn't know anything about GLFW. Meanwhile, SceneRunner is the class that actually owns the GLFWwindow*.

Right now, your userInput() method expects a GLFWwindow*:

```
void SceneBasic_Uniform::userInput(GLFWwindow* WindowIn)
```

...but Scene::update(float t) has no way to pass that window pointer down because the base class has no knowledge of GLFW.

There are a few clean ways to solve this:

Option 1: Pass GLFWwindow* into update

Change your Scene class to allow an optional GLFWwindow* parameter in update:

```
class Scene {  
public:  
    virtual void update(float t, GLFWwindow* window = nullptr) = 0;  
};
```

Then in SceneBasic_Uniform:

```
void SceneBasic_Uniform::update(float t, GLFWwindow* window)  
{  
    deltaTime = t - tPrev;  
    if (tPrev == 0.0f) deltaTime = 0.0f;  
    tPrev = t;  
  
    // Handle input  
    if (window) {  
        userInput(window);  
    }  
}
```

And in SceneRunner::mainLoop:

```
scene.update(float(glfwGetTime()), window);
```

Skybox Shader

I have a basic skybox rendered on the scene, but it is currently interfering with lighting?

1. Stop sampling the cubemap in basic_uniform.frag

Delete this from the lighting shader entirely:

```
layout (binding = 4) uniform samplerCube SkyBoxTexture;
```

```
in vec3 VecPosition;
```

And especially this:

```
FragColor = vec4(texture(SkyBoxTexture, normalize(VecPosition)).rgb, 1.0);
```

Your lighting shader should **never know the skybox exists**.

2. Use a dedicated skybox shader

You already have skybox(100.0f) — good.

Now it needs its *own* shader.

Skybox vertex shader

```
#version 460
```

```
layout (location = 0) in vec3 VertexPosition;
```

```
out vec3 TexDir;
```

```
uniform mat4 VP;
```

```
void main()
```

```
{
```

```
    TexDir = VertexPosition;
```

```
    gl_Position = VP * vec4(VertexPosition, 1.0);  
}
```

Skybox fragment shader

```
#version 460  
  
in vec3 TexDir;  
  
layout (binding = 4) uniform samplerCube SkyBoxTexture;  
  
out vec4 FragColor;  
  
void main()  
{  
    FragColor = vec4(texture(SkyBoxTexture, normalize(TexDir)).rgb, 1.0);  
}
```

3. Render the skybox with camera translation removed

This part is crucial and currently missing.

```
// Render skybox FIRST
```

```
glDepthMask(GL_FALSE);
```

```
glDepthFunc(GL_LEQUAL);
```

```
skyboxProg.use();
```

```
mat4 viewNoTranslation = mat4(mat3(view));
```

```
mat4 vp = projection * viewNoTranslation;
```

```
skyboxProg.setUniform("VP", vp);
```

```
glActiveTexture(GL_TEXTURE4);
```

```
glBindTexture(GL_TEXTURE_CUBE_MAP, skyboxTexture);
```

```
skybox.render();
```

```
glDepthMask(GL_TRUE);
```

```
glDepthFunc(GL_LESS);
```

Skybox Inverted

For some reason my skybox is inverted, with the side face upside down and top/bottom flipped. How can I flip the sides and swap the top/bottom?

in your **skybox fragment shader**:

```
vec3 flippedVec = VecPosition; // Compensate for coordinate system difference if
(abs(VecPosition.y) > abs(VecPosition.x) && abs(VecPosition.y) > abs(VecPosition.z)) { //
Top/bottom faces flippedVec = vec3(VecPosition.x, VecPosition.y, -VecPosition.z); } else
{ // Side faces - flip Y flippedVec = vec3(VecPosition.x, -VecPosition.y, VecPosition.z); }
```

Bloom Bindings

I've implemented bloom onto my HDR rendering, the problem is currently the blur textures are binding in the slot for my normal and mix textures. Which bindings should I use in each pass

Your HDR + bloom logic is mostly fine.

The real issue is here:

● The Core Problem

In your fragment shader:

```
layout (binding = 0) uniform sampler2D HDRTex;
```

```
layout (binding = 1) uniform sampler2D BlurTex1;
```

```
layout (binding = 2) uniform sampler2D BlurTex2;
```

These samplers are **explicitly bound** to texture units 0, 1, and 2.

But in your scene rendering you are also using:

```
GL_TEXTURE0 → diffuse textures
```

```
GL_TEXTURE1 → normal textures
```

```
GL_TEXTURE2 → mix textures
```

```
GL_TEXTURE3 → mix normal textures
```

So when you reach **Pass5**, texture units 0/1/2 are still bound to:

- barrel diffuse
- barrel normal
- damage diffuse
etc.

That means in Pass5:

```
vec4 color = texture(HDRTex, TexCoord); // unit 0
```

```
vec4 blurTex = texture(BlurTex2, TexCoord); // unit 2
```

You are sampling whatever texture is currently bound to unit 0 and 2.

☞ Which at that moment is your **barrel normal map**.

That's why you see the normal map overlaid fullscreen.

☑ Why It Only Appeared After Adding Bloom

Before:

```
return toneMappedColor;
```

Even if HDRTex was wrong, it may have coincidentally still been bound correctly.

After:

```
return toneMappedColor + blurTex;
```

Now you're also sampling BlurTex2 (unit 2), which is absolutely still bound to your barrel/mix textures.

So it becomes visible.

☑ The Correct Fix

You must explicitly bind the correct textures before Pass2–Pass5.

✓ Fix Option 1 (Best): Explicitly Bind HDR & Blur Textures Before Quad Rendering

In pass2(), pass3(), pass4(), and pass5(), bind the correct textures:

Example: Fix Pass5

```
void SceneBasic_Uniform::pass5()
{
    prog.setUniform("Pass", 5);
    glBindFramebuffer(GL_FRAMEBUFFER, 0);

    glClear(GL_COLOR_BUFFER_BIT);
    glViewport(0, 0, width, height);

    // 💧 Bind HDR texture to unit 0
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, hdrTexture);

    // 💧 Bind blurred bloom texture to unit 2
    glActiveTexture(GL_TEXTURE2);
    glBindTexture(GL_TEXTURE_2D, tex1); // final blurred result

    glBindSampler(1, linearSampler);

    glBindVertexArray(quad);
    glDrawArrays(GL_TRIANGLES, 0, 6);

    glBindSampler(1, nearestSampler);
}
```

✓ Also Fix Pass2

In pass2 (bright pass), you must bind `hdrTexture` to unit 0:

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, hdrTexture);
```

✓ Fix Pass3

Bind tex1 as input:

```
glActiveTexture(GL_TEXTURE1);  
glBindTexture(GL_TEXTURE_2D, tex1);
```

✓ Fix Pass4

Bind tex2 as input:

```
glActiveTexture(GL_TEXTURE2);  
glBindTexture(GL_TEXTURE_2D, tex2);
```

Ignore Skybox in Average Luminance

How can I prevent AvgLum from being too high and causing super bright areas, especially from skybox

You can simplify Pass5() and implement proper tone mapping **per-pixel**:

```
vec4 Pass5() {  
    vec3 hdrColor = texture(HDRTex, TexCoord).rgb;  
    vec3 bloom = texture(BlurTex1, TexCoord).rgb;  
  
    // Compute exposure factor  
    float exposureFactor = Exposure / (AvgLum + 0.0001);  
  
    // Apply exposure  
    vec3 mapped = hdrColor * exposureFactor;  
  
    // Simple Reinhard tone mapping  
    mapped = mapped / (mapped + vec3(1.0));  
  
    // Optional: add bloom
```

```
mapped += bloom;

// Gamma correction
mapped = pow(mapped, vec3(1.0 / Gamma));

return vec4(mapped, 1.0);
}
```